

Documentación de implementación para la aplicación Pociones de Minecraft

Índice

1. Introducción	3
2. Arquitectura general del proyecto	3
3. Patrones de diseño utilizados	3
3.1. Patrón Singleton	3
3.1.1. ¿Qué es?	3
3.1.2. ¿Cómo lo utilicé?	3
3.1.3. ¿Para qué lo utilicé?	4
3.2. Patrón Repository	4
3.2.1. ¿Qué es?	4
3.2.2. ¿Cómo lo utilicé?	4
3.2.3. ¿Para qué lo utilicé?	4
3.3. Patrón DTO (Data Transfer Object)	4
3.3.1. ¿Qué es?	4
3.3.2. ¿Cómo lo utilicé?	5
3.3.3. ¿Para qué lo utilicé?	5
3.4. Patrón Builder	5
3.4.1. ¿Qué es?	5
3.4.2. ¿Cómo lo utilicé?	5
3.4.3. ¿Para qué lo utilicé?	6
3.5. Patrón Dependency Injection	6
3.5.1. ¿Qué es?	6
3.5.2. ¿Cómo lo utilicé?	6
3.5.3. ¿Para qué lo utilicé?	6
4. Buenas prácticas implementadas	6
4.1. Separación de responsabilidades	6
4.2. Uso de JWT	7
4.3. Encriptación de contraseñas	7
4.4. Uso de HTTP Status Codes	7
4.5. Uso de interfaces	7

5. Seguridad implementada	7
5.1. Spring Security	7
5.2. Control de sesiones	8
5.3. Configuración CORS	8
6. Base de datos	8
7. Despliegue	8
8. Conclusión	9

1. Introducción

En este proyecto desarrollé la aplicación **Pociones de Minecraft**, utilizando una arquitectura cliente-servidor compuesta por Flutter para el frontend, Spring Boot para el backend y PostgreSQL como sistema gestor de base de datos.

El objetivo principal de la aplicación es permitir a los usuarios consultar y administrar recetas de pociones inspiradas en Minecraft, además de implementar funcionalidades como autenticación, comentarios y reacciones.

Durante el desarrollo implementé distintos patrones de diseño y buenas prácticas de programación orientada a objetos con el propósito de mantener una arquitectura organizada, escalable y reutilizable.

2. Arquitectura general del proyecto

Para organizar correctamente el sistema, dividí el proyecto en distintas capas:

- Frontend Flutter
- Backend Spring Boot
- API REST
- Base de datos PostgreSQL

La arquitectura sigue el flujo:

UI → Controladores → Servicios → Repositorios → Base de datos

Con esta separación logré desacoplar responsabilidades y facilitar el mantenimiento del sistema.

3. Patrones de diseño utilizados

3.1. Patrón Singleton

3.1.1. ¿Qué es?

El patrón Singleton garantiza que únicamente exista una instancia de una clase durante toda la ejecución del programa.

3.1.2. ¿Cómo lo utilicé?

Lo implementé dentro del servicio de autenticación en Flutter:

```
1 class AuthService implements IAuthRepository {
2     static final AuthService _instance = AuthService._internal();
3
4     AuthService._internal();
5
6     factory AuthService() {
```

```

7     return _instance;
8 }
9 }

```

3.1.3. ¿Para qué lo utilicé?

Utilicé este patrón para:

- Mantener una única sesión de autenticación
- Evitar múltiples instancias del cliente HTTP
- Compartir el token JWT en toda la aplicación

3.2. Patrón Repository

3.2.1. ¿Qué es?

El patrón Repository permite encapsular el acceso a datos y separar la lógica de persistencia de la lógica de negocio.

3.2.2. ¿Cómo lo utilicé?

Lo implementé utilizando Spring Data JPA:

```

1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> {
3
4     Optional<User> findByUsername(String username);
5
6     Boolean existsByEmail(String email);
7 }

```

3.2.3. ¿Para qué lo utilicé?

Con este patrón logré:

- Abstraer consultas SQL
- Facilitar operaciones CRUD
- Mantener desacoplado el acceso a datos

3.3. Patrón DTO (Data Transfer Object)

3.3.1. ¿Qué es?

El patrón DTO se utiliza para transferir información entre capas sin exponer directamente las entidades de la base de datos.

3.3.2. ¿Cómo lo utilicé?

Lo implementé en las clases de autenticación:

```
1 public class LoginRequest {
2
3     private String username;
4
5     private String password;
6 }
7
8 public class JwtResponse {
9
10    private String accessToken;
11
12    private Long id;
13
14    private String username;
15 }
```

3.3.3. ¿Para qué lo utilicé?

Este patrón me permitió:

- Validar la información recibida
- Proteger entidades internas
- Mejorar seguridad y organización

3.4. Patrón Builder

3.4.1. ¿Qué es?

El patrón Builder facilita la creación de objetos complejos.

3.4.2. ¿Cómo lo utilicé?

Lo implementé en la clase UserDetailsImpl:

```
1 public static UserDetailsImpl build(User user) {
2
3     List<GrantedAuthority> authorities =
4         user.getRoles().stream()
5             .map(role -> new SimpleGrantedAuthority(
6                 role.getName().name()))
7             .collect(Collectors.toList());
8
9     return new UserDetailsImpl(
10         user.getId(),
11         user.getUsername(),
12         user.getEmail(),
13         user.getPassword(),
```

```
14     authorities);  
15 }
```

3.4.3. ¿Para qué lo utilicé?

Lo utilicé para:

- Construir objetos de autenticación complejos
- Centralizar la creación de usuarios autenticados
- Mejorar reutilización del código

3.5. Patrón Dependency Injection

3.5.1. ¿Qué es?

La inyección de dependencias permite que los objetos sean proporcionados automáticamente por el framework.

3.5.2. ¿Cómo lo utilicé?

Implementé este patrón en controladores y servicios:

```
1 @Autowired  
2 AuthenticationManager authenticationManager;  
3  
4 @Autowired  
5 UserRepository userRepository;
```

3.5.3. ¿Para qué lo utilicé?

Gracias a este patrón pude:

- Reducir acoplamiento
- Facilitar pruebas
- Mejorar mantenibilidad

4. Buenas prácticas implementadas

4.1. Separación de responsabilidades

Organicé el sistema para que cada clase tuviera una única responsabilidad:

- Los Controllers manejan peticiones HTTP
- Los Services contienen lógica de negocio
- Los Repositories acceden a la base de datos
- Los Models representan entidades

4.2. Uso de JWT

Implementé autenticación mediante JWT.

```
1 String jwt = jwtUtils.generateJwtToken(authentication);
```

Esto me permitió:

- Utilizar autenticación stateless
- Mejorar seguridad
- Facilitar integración con Flutter

4.3. Encriptación de contraseñas

Protegí las contraseñas utilizando BCrypt.

```
1 encoder.encode(signUpRequest.getPassword())
```

Con esto evité almacenar contraseñas en texto plano.

4.4. Uso de HTTP Status Codes

Utilicé códigos HTTP adecuados para representar estados del sistema:

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized

4.5. Uso de interfaces

En Flutter implementé interfaces para desacoplar servicios:

```
1 class AuthService implements IAuthRepository
```

Esto me permitió:

- Mejorar escalabilidad
- Facilitar pruebas
- Permitir futuras implementaciones

5. Seguridad implementada

5.1. Spring Security

Utilicé Spring Security para proteger endpoints.

```
1 .requestMatchers("/api/auth/**").permitAll()  
2 .anyRequest().authenticated()
```

5.2. Control de sesiones

Configuré el sistema para trabajar con sesiones stateless.

```
1 .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

5.3. Configuración CORS

Configuré CORS para permitir comunicación segura entre Flutter Web y Spring Boot.

```
1 configuration.setAllowedOrigins(List.of(  
2     "http://localhost:3000"  
3 ));
```

6. Base de datos

Utilicé PostgreSQL hospedado en Aiven como sistema gestor de base de datos. Las principales entidades implementadas fueron:

- User
- Role
- Potion
- PotionComment
- PotionReaction

También implementé relaciones:

- OneToMany
- ManyToMany
- ManyToOne

7. Despliegue

Realicé el despliegue del backend utilizando Render. Las tecnologías utilizadas fueron:

- Spring Boot
- PostgreSQL
- Aiven
- Render

8. Conclusión

Durante el desarrollo de la aplicación Pociones de Minecraft implementé distintos patrones de diseño y buenas prácticas con el objetivo de construir una arquitectura organizada, reutilizable y escalable.

El uso conjunto de Flutter y Spring Boot me permitió desarrollar una aplicación moderna basada en APIs REST, autenticación JWT y separación adecuada de responsabilidades.